ELEC 475 Lab 3, Alistair Barfoot and Luke Barry

Contents

1 Network Architecture

The backbone for this network architecture is the *MobileNetV3-small* pretrained model from the *torchvision.models* library. This backbone is split up into three sequential parts: the low-level features (fine-grained details), the mid-level features (intermediate representations), and the high-level features (semantic information).

A context module is added after the backbone with three branches each with dilation rates of 1, 6, and 12 respectively. It is projected to 128 channels and includes a 2D dropout with p=0.1.

Depthwise seperable convolution is used for the low and mid level features. This consists of a depthwise convolution, then a pointwise convolution, followed by batch normalization and a ReLU activation.

After the depthwise seperable convolutions, each of the three feature levels are upsampled to the same size as the low-level features using bilinear interpolation, and then concatenated together. They are then sent through a decoder convolution block which uses two 3x3 convolutions, batch normalization and a ReLU function.

Finally, the image is upsampled to the original size using bilinear interpolation, at which point dropout with p=0.1 is applied and a 1x1 convolution which maps the output to one of 21 classes.

2 Knowledge Distillation

For the knowledge distillation for this lab, the FCN-ResNet50 model was used as the teacher network, and the MBV3SmallSeg model (MobileNetV3-Small backbone) was used as the student network. For this lab, two different types of knowledge distillation were used: response-based distillation and feature-based distillation.

For response-based distillation, the total loss combined two components: the ground truth loss and the distillation loss.

For this method, the image is passed through both the teacher and student networks to obtain each of their logits. Each of the logits are scaled used a temperature parameter T and passed through a softmax function. The loss is then calculated and sent backwards through the student network. The loss is calculated as follows:

$$\mathcal{L}_{total} = \alpha \cdot \mathcal{L}_{GT} + \beta \cdot \mathcal{L}_{KD} \tag{1}$$

For feature-based distillation, the logits from the mid-level features of both the teacher and student networks are extracted. The loss is calculated by finding the angle between the two

feature maps using the cosine law. This is calculated as follows:

$$\mathcal{L}_{feature} = 1 - \frac{F_{student} \cdot F_{teacher}}{||F_{student}|| \cdot ||F_{teacher}||} \tag{2}$$

and the total loss is thus calculated as:

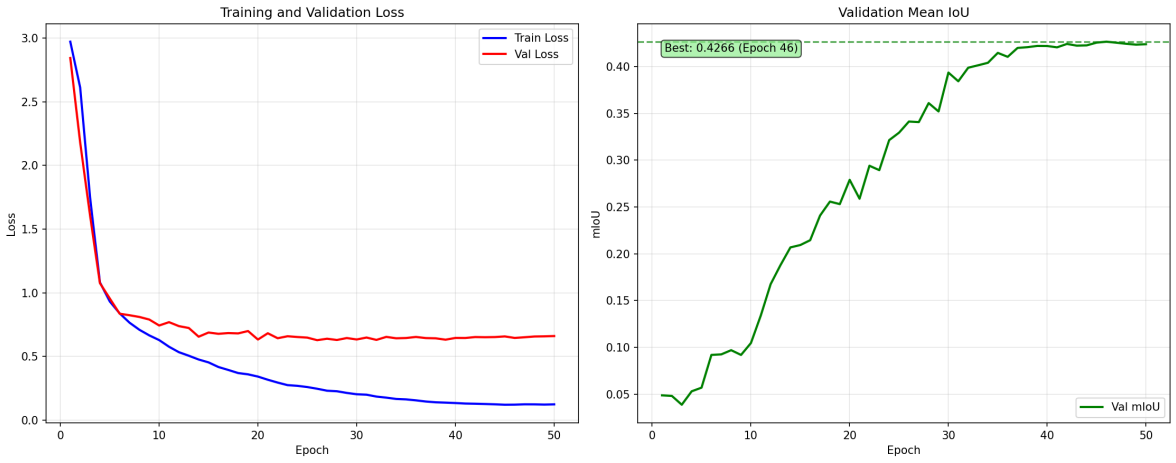
$$\mathcal{L}_{total} = \alpha \cdot \mathcal{L}_{CE} + \beta \cdot \mathcal{L}_{\text{feature}} \tag{3}$$

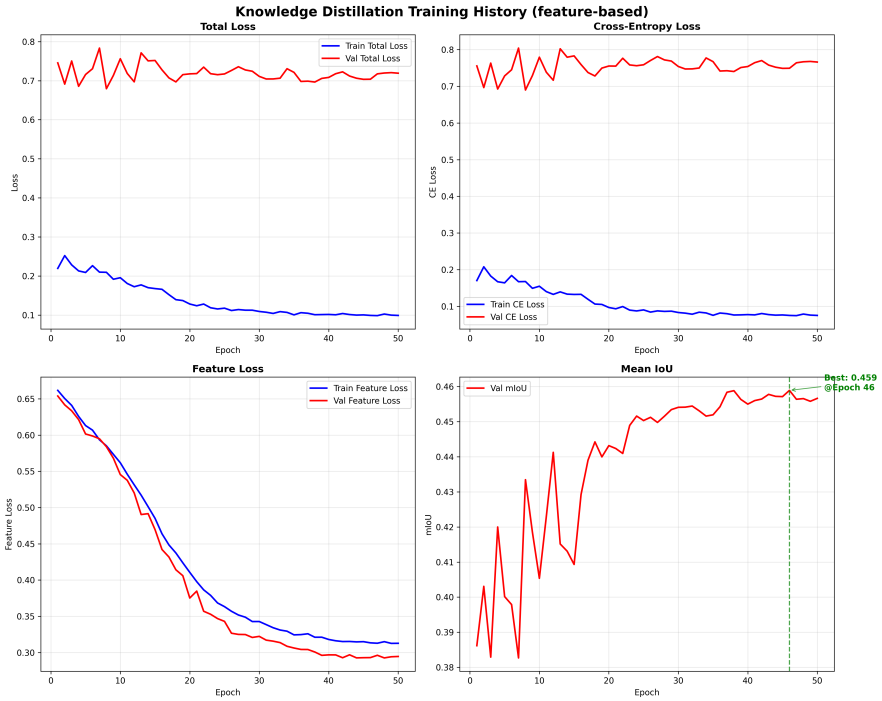
3 Hyperparameters

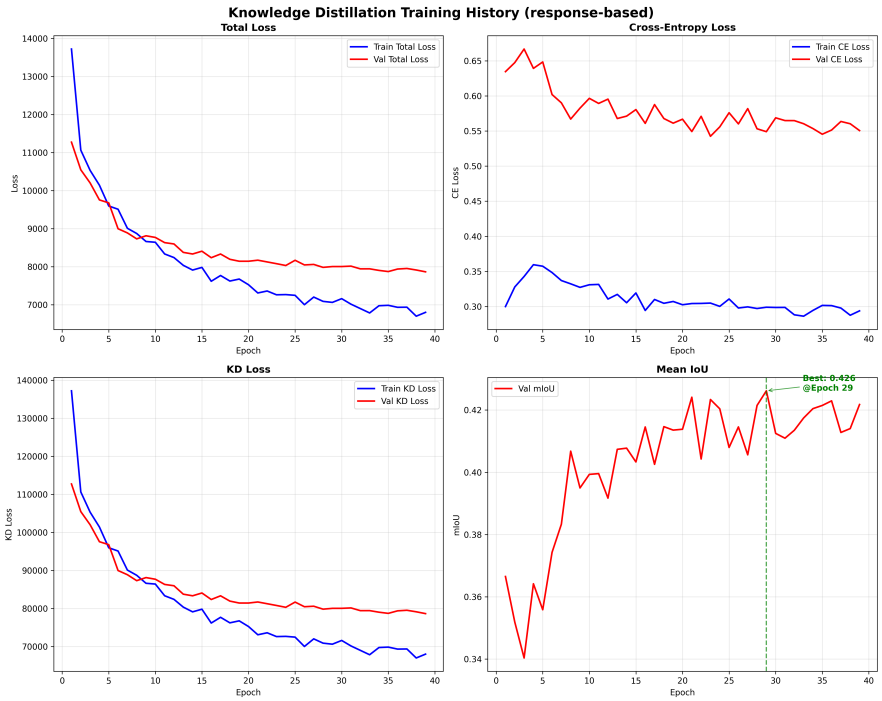
All of the models were trained for 50 epochs with a patience of 10 epochs for early stopping. The learning rate was set to $1e-3$ for the non-KD model and $1e-4$ for the KD models. The batch size was set to 16 for all models. The optimizer used was AdamW with a weight decay of $1e-4$.

For the knowledge distillation models, the temperature T was set to 3.5 and the weights α and β were set to 0.9 and 0.1 respectively for both response-based and feature-based distillation.

The loss and mIoU graphs are as follows:




Knowledge Distillation Training History (feature-based)
Total Loss
Train Total Loss
Val Total Loss
Loss
Epoch
Cross-Entropy Loss
Train CE Loss
Val CE Loss
CE Loss
Epoch
Feature Loss
Train Feature Loss
Val Feature Loss
Feature Loss
Epoch
Mean IoU
Val mIoU
Best: 0.459
@Epoch 46
mIoU
Epoch


Knowledge Distillation Training History (response-based)
Total Loss
Train Total Loss
Val Total Loss
Loss
Epoch
Cross-Entropy Loss
Train CE Loss
Val CE Loss
CE Loss
Epoch
KD Loss
Train KD Loss
Val KD Loss
KD Loss
Epoch
Mean IoU
Val mIoU
Best: 0.426
@Epoch 29
mIoU
Epoch

4 Results

The hardware used was a NVIDIA 3060 Super Laptop Edition with 6 GB of VRAM. Our experiments started extremely poorly with the best mIoU reaching at best 5% after 10 epochs and failing to improve. A test was made to ensure all parts of initialization were working well. There was a server initialization problem as the initial logits had a range $\begin{bmatrix} -8595 & 7014 \end{bmatrix}$. After fixing the Initial logits the mIoU improved to 42% after 47 epochs. The model not only worked as intended it worked very well for a model with around 1 million parameters.

The time per epoch for the original model without knowledge distillation was 107 seconds. For both the Response-based and Feature-based knowledge distillation the average time per epoch was 134 seconds.

Table 1: Knowledge Distillation Results

Knowledge Distillation	**mIoU**	**# Parameters**	**Inference Speed (ms/sample)**
Without	0.426	1414805	3.97
Response-Based	0.426	1414805	3.79
Feature-Based	0.459	1414805	3.49

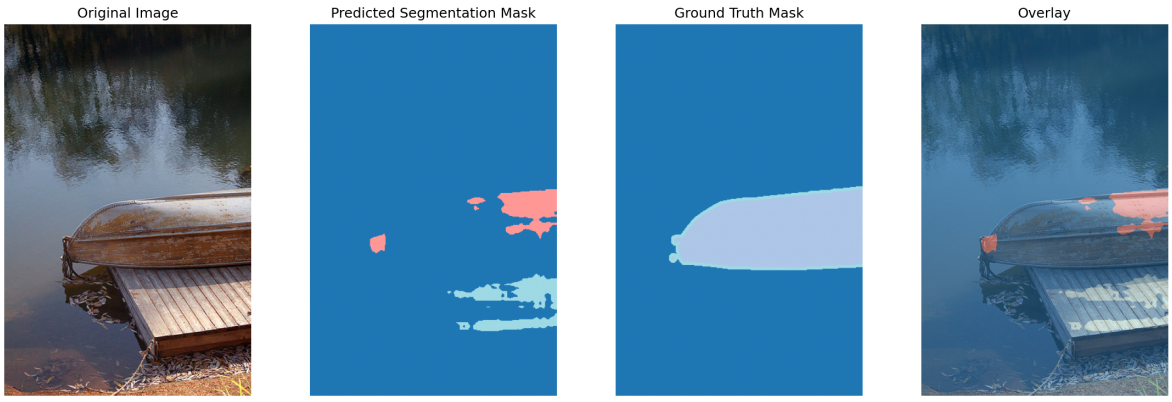


Figure 1: Segmentation Result with Improper Initialization

Figure 2: Sample Segmentation Results after Training without KD

5 Performance

The performance of the MobileNetV3 custom system was overall successful. The system without knowledge distillation was already quite good with an mIoU of 42%. The knowledge distillation started off creating a model with a worse mIoU of that of the original model. The first epoch had an mIoU of 30% with a warm start and that created confusion as to if the model was working correctly. A run was done without any pretrained weights but ended up with a loss higher than that of the warm start. The response based knowledge distillation ultimately did the same as the model without any knowledge distillation, achieving a score of 42% mIoU. The feature-based knowledge distillation did help the model by 4% in mIoU. It makes sense that feature-based distillation did better since it gives the student model a more full understanding of the teacher-model, not just the outputs.

Table 2: Hyperparameter Tuning Results

Attempts	**Initialization**	**Hyperparameters**	**mIoU**	**mIoU (epoch 1)**
1	Warm Start	$\alpha = 0.6, \beta = 0.4, lr = 1e-3$	0.40	0.31
2	Cold Start	$\alpha = 0.6, \beta = 0.4, lr = 1e-3$	0.36	0.08
3	Warm Start	$\alpha = 0.8, \beta = 0.2, lr = 5e-4$	0.41	0.32
4	Warm Start	$\alpha = 0.9, \beta = 0.1, lr = 5e-4$	0.45	0.36

6 LLM Usage

We used Claude Sonnet 4 for the duration of the project in GitHub Copilot. Throughout the entire project, code verification scripts were set up alongside changes made to the code by the LLM. This ensured all new changes worked correctly before implementing them into time-intensive training. We fed the LLM PyTorch documents to ensure its model loading and handling were robust. Oftentimes we questioned the LLM about its own decisions, especially ones which were unclear to our group. This allowed us to gain a better understanding of the decisions being made. This stopped the model from implementing something that is not understood by us while also being incorrect. We provided real data and context to the model to ensure that it wouldn't hallucinate, we would give the model graphs and values along with concrete parameters instead of asking for generic advice. Ultimately, every suggestion given by Claude was immediately tested and prevented Claude from building on false expectations or generating solutions that wouldn't work in this specific context.